All-pairs shortest paths

Part VI Graph Algorithms

Scope

All-pairs shortest paths uses two distinct DP states, edge budgets and intermediate vertices. Min-plus products, Floyd–Warshall, and Johnson connect path reconstruction, zero cycles, affected pairs, and the signs and costs of reweighting.

Prerequisites

Min-plus matrices and predecessor representations.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Chapter route

Learning objectives

- Connect min-plus multiplication to shortest paths.
- Derive Floyd–Warshall from intermediate-vertex states.
- Use Johnson reweighting on sparse graphs with negative edges.
- 25.1 Shortest paths and matrix multiplication
- 25.2 The Floyd-Warshall algorithm
- 25.3 Johnson’ s algorithm for sparse graphs

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

25.1 Shortest paths and matrix multiplication

APSP takes a directed weighted graph on n vertices, allowing negative edges, and returns a distance for every pair(i, j). Unreachable pairs have$+\infty$. If i can reach a negative cycle that can reach j, repeating it makes walk cost unbounded below. Define that distance as$-\infty$; no finite shortest path exists for the pair.

First assume no negative cycle. Every repeated-vertex walk contains a nonnegative cycle, which can be removed without increasing cost. Repeating this leaves a simple path. For$i \neq j$, a shortest path therefore uses at most$n-1$edges. For i=j, the zero-length path costs 0.

Matrix W[i, j] stores the lightest direct edge, or∞if absent. Set W[i, i]=0 to allow no edge. For negative-self-loop detection, initialize to min(0, w(i, i)) rather than overwriting a negative loop with 0. All diagonals in the following cycle-free example are 0.

State as an upper bound on edge count

Let$L^{(m)}[i, j]$be the minimum weight of a walk with **at most m edges**, not exactly m. Set$L^{(0)}=I$, with zero diagonal and∞elsewhere. Splitting at the last edge gives

$L^{(m+1)}[i, j]=\min_k(L^{(m)}[i, k]+W[k, j])$.

Every nonempty candidate splits into an at-most-m-edge prefix and its last edge; every finite recurrence term conversely concatenates to a valid walk. Shorter old paths remain through k=j and W[j, j]=0. Both directions describe the same feasible walks. With negative cycles, a fixed finite edge budget still defines a meaningful finite optimum, but budget$n-1$no longer gives unrestricted distances.

Define$(A \otimes B)[i, j]=\min_k(A[i, k]+B[k, j])$. Matrices for budgets a, b concatenate to budget a+b. Conversely, split any at-most-(a+b)-edge walk after min(a, its length) edges; the suffix uses at most b. Hence$L^{(a)} \otimes L^{(b)}=L^{(a+b)}$.

Min-plus algebra and two algorithms

Associativity expands both sides to the same finite minimum:$((A \otimes B) \otimes C)[i, j] = \min_{p,\varphi}(A[i, p] + B[p, q] + C[q, j]) = (A \otimes (B \otimes C))[i, j]$. Matrix I is the identity;∞ is neutral for min and$\infty + a$ finite value$= \infty$. Do not introduce$-\infty$ at this stage, since$\infty + (-\infty)$ is undefined.

```
Extend(A, B):
  C=new n×n matrix filled with∞
  for i, j, k: C[i, j]=min(C[i, j], A[i, k]+B[k, j])
  return C
Slow(W): L=I; repeat n−1 times: L=Extend(L, W); return L
Square(W): L=W; m=1
  while m<n−1: L=Extend(L, L); m=2m
  return L
```

Slow performs $\Theta(n)$ products of $\Theta(n^3)$, totaling $\Theta(n^4)$ for $n \geq 2$. Square performs $\text{ceil}(\log_2(n-1))$ squarings, taking $O(n^3 \log n)$. Two input matrices and a fresh output need $O(n^2)$ working space. Overwriting inputs during a product invalidates its fixed old-budget interpretation. For n=0 return an empty matrix; handle n=1 directly through its diagonal.

Min-plus algebra and two algorithms (continued)

Ordinary multiplication replaces min by addition and addition by multiplication, but Strassen's subtraction formulas do not directly apply to min-plus. The analogy does not transfer fast multiplication bounds for rings.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

One four-vertex, six-edge example

All matrices, Floyd stages, and Johnson states below use1→2:4,1→3:7,2→3:−2,2→4:6,3→4:3,4→2:1. Vertex 1 has no incoming edge. Cycle 2→3→4→2 costs 2, so there is no negative cycle.

Rows of W are[0,4,7,∞],[∞,0,−2,6],[∞,∞,0,3],[∞,1,∞,0].

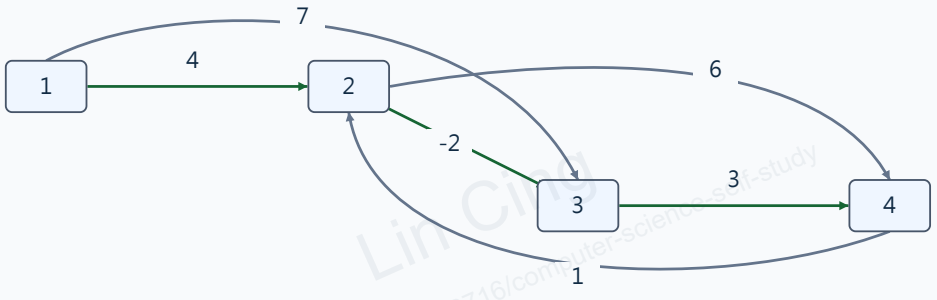
Rows of $L^{(2)}$ are[0,4,2,10],[∞,0,−2,1],[∞,4,0,3],[∞,1,−1,0].

For example $L^{(2)}[1,4]=\min(0+\infty,4+6,7+3,\infty+0)=10$, with two tied two-edge paths. Also $L^{(2)}[2,4]=-2+3=1$. Only the third extension permits 1→2→3→4 and gives $L^{(3)}[1,4]=5$; all other entries stay unchanged. Since n=4, $L^{(3)}$ is the answer, as is the squared $L^{(4)}$.

One four-vertex, six-edge example (continued)

Six directed edges and edge-budgeted paths

Green: best 3-edge route. Gray: other input edges. Labels: original weights.



1→2→4=10 (2 edges); 1→3→4=10 (2 edges); 1→2→3→4=5 (3 edges).

25.2 The Floyd-Warshall algorithm

Here k bounds the **identifiers of allowed intermediate vertices**, not edge count. State $D^{(k)}[i, j]$ is the minimum walk cost when all intermediate vertices belong to$\{1,\ldots, k\}$; endpoints i, j are unrestricted. Without negative cycles, choose a simple path, so k occurs at most once.

An optimum avoiding k belongs to old state $D^{(k-1)}$. An optimum using k splits there into segments with no internal k, giving $D^{(k-1)}[i, k]+D^{(k-1)}[k, j]$. For optimal substructure, replacing a nonoptimal segment with a cheaper legal segment improves the concatenation; deleting any nonnegative cycles cannot worsen it, contradicting optimality. Conversely, concatenating two old-state paths introduces only intermediate k and cannot give an illegally low candidate.

Thus $D^{(0)}=W$ and $D^{(k)}[i, j]=\min(D^{(k-1)}[i, j], D^{(k-1)}[i, k]+D^{(k-1)}[k, j])$. At k=n all intermediates are allowed. New matrices per stage take $\Theta(n^3)$ time. Saving all stages uses $\Theta(n^3)$ space; retaining only old and new uses $\Theta(n^2)$.

In-place updates work, arbitrary loop orders do not

Without negative cycles, D[k, k]=0 at every stage. Updating row k compares D[k, j] with D[k, k]+D[k, j], leaving it unchanged; column k is likewise unchanged. All stage-k inputs D[i, k], D[k, j] therefore remain old-stage values. Updating each D[i, j] once exactly implements the recurrence in one matrix.

```
D=copy(W)
for k=1..n:
  for i=1..n:
    for j=1..n:
      if D[i, k]!=∞ and D[k, j]!=∞:
        D[i, j]=min(D[i, j], D[i, k]+D[k, j])
```

Time is $\Theta(n^3)$, space $\Theta(n^2)$. Fixed-width implementations must prevent an INF sentinel plus a negative number from becoming falsely reachable and must avoid overflow. Check reachability before addition; the mathematical theorem assumes exact arithmetic.

In-place updates work, arbitrary loop orders do not (continued)

For a counterexample, use unit-weight chain1→2→4→3. A single i, j, k pass considers i=1, j=3 before D[1,4] or D[2,3] is finite, so D[1,3] remains∞. Later j=4 computes D[1,4]=2 but never revisits j=3. Correct k, i, j order gives D[1,3]=3.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

Complete matrices at every k

Rows and columns use1,2,3,4. $D^{(0)}$=W; vertex 1 has no incoming edge, so $D^{(1)}$=W too.

Stage	Row1	Row2	Row3	Row4
$D^{(2)}$	0,4,2,10	∞,0,−2,6	∞,∞,0,3	∞,1,−1,0
$D^{(3)}$	0,4,2,5	∞,0,−2,1	∞,∞,0,3	∞,1,−1,0
$D^{(4)}$	0,4,2,5	∞,0,−2,1	∞,4,0,3	∞,1,−1,0

Stage 2 changes(1,3) to 4−2=2,(1,4) to 4+6=10, and(4,3) to 1−2=−1. Stage 3 changes(1,4) to 2+3=5 and(2,4) to−2+3=1. Stage 4 changes(3,2) to 3+1=4. Every other old value is at most its candidate and remains.

Unlike $D^{(2)}$, $L^{(2)}$ already permits 2→3→4 and 3→4→2. Intermediate vertices 3,4 are not allowed in $D^{(2)}$, whose corresponding entries remain6 and∞. A second step in each algorithm means a different state.

Complete matrices at every k (continued)

Six Floyd updates: two old inputs give one new value

Strict updates only. Inputs use the previous stage; unlisted entries stay unchanged.

k	(i, j)	Old	D[i, k]	D[k, j]	New
2	(1,3)	7	4	-2	2
2	(1,4)	∞	4	6	10
2	(4,3)	∞	1	-2	-1
3	(1,4)	10	2	3	5
3	(2,4)	6	-2	3	1
4	(3,2)	∞	3	1	4

Green: new minimum. No updates at k=1; three at k=2, two at k=3, one at k=4.

A predecessor is not an intermediate vertex

Entry Π[i, j] is the vertex **immediately preceding j** on a selected shortest i-to-j path, not arbitrary intermediate k. Initialize Π[i, j]=i for direct edges; use NIL for diagonals and unreachable entries. With two stage buffers, a strict improvement through k assigns Πnew[i, j]=Πold[k, j], because the final edge belongs to the k-to-j suffix. Otherwise preserve the old predecessor. When k=j, no strict improvement occurs, so a zero-length suffix cannot overwrite a valid predecessor with NIL.

Preserve the old route on equal costs; changing $<$ to $\leq$ is unsafe. With only edge 1→2 of weight 1, stage k=2 has equal candidate1+0=1. Assigning Π[1,2]←Π[2,2]=NIL then destroys the existing route. Zero cycles likewise prevent arbitrary tight-edge parent choices without an acyclicity argument.

In the example, Π[1,4] first improves at k=2 to Π[2,4]=2, giving 1→2→4. At k=3 it becomes Π[3,4]=3. With Π[1,3]=2 and Π[1,2]=1, the final reverse chain is 4←3←2←1, cost 4−2+3=5. The next tie rule provides a directly decreasing integer measure for a complete predecessor tree.

Provably acyclic predecessors even with zero cycles

Store pair(cost, hops), minimizing cost first and then hops. Concatenation adds both coordinates. Initialize diagonals(0,0), direct edges(w,1), and absent paths(∞,∞), then apply the same Floyd recurrence. Without negative cycles, positive cycles worsen cost and zero cycles worsen hops. A lexicographic optimum has a simple representative, so the decomposition and in-place proofs still hold. The algorithm uses $\Theta(n^3)$ operations on fixed-size pairs.

For each fixed source s and finite $v \neq s$, choose an original incoming edge $u \to v$ satisfying

$cost[s, v]=cost[s, u]+w(u, v)$, $hops[s, v]=hops[s, u]+1$.

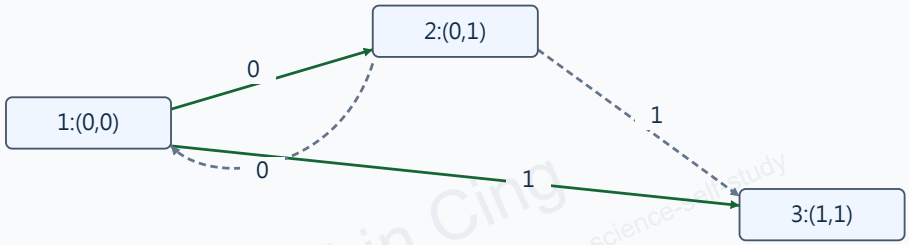
Such a u exists on the final edge of a lexicographically optimal path. A nonoptimal prefix could be replaced to improve the whole path, a contradiction. Set $\Pi[s, v]=u$. Every parent step decreases nonnegative integer hops by one, excluding cycles and infinite descent. All finite nonsource vertices have parents, so chains terminate at s and reconstruct actual shortest paths.

Scanning all m edges for every source costs $O(nm)$, no more than $\Theta(n^3)$ on a simple graph with $m \leq n^2$. Saved cost, hops,Π need $\Theta(n^2)$ space. For zero edges $1 \leftrightarrow 2$ and weight 1 edges $1 \to 3, 2 \to 3$, source 1 has hops=(0,1,1). It cannot acquire parent2 and no$1 \leftrightarrow 2$ parent cycle arises.

Provably acyclic predecessors even with zero cycles (continued)

Safe backtracking through zero cycles: hops decrease

Boxes: vertex:(cost, hops). Green: selected edges; backtrack against their arrows.



$\Pi[1,2]=1$, $\Pi[1,3]=1$. Source 1 has no parent.

Route $2 \to 3$ also gives cost 1 but needs 2 hops, losing to direct$1 \to 3$ with 1 hop.

Gray dashed: unselected. Every parent step lowers hops by 1, excluding zero-cycle parents.

Negative-cycle detection and affected pairs

If final Floyd has D[k, k]<0, its finite value concatenates actual edges into a negative closed walk. Decomposing this into simple cycles finds at least one negative cycle. Conversely, take a negative simple cycle and its largest-numbered vertex k. All other vertices are allowed intermediates of its k-to-k walk, so Floyd detects a negative closed walk and some negative diagonal. This proves both detection directions.

With negative cycles, fixed stages produce some finite walk values, which are not shortest distances for affected pairs. The earlier in-place equivalence proof relying on D[k, k]=0 no longer applies. Standard in-place k, i, j still detects cycles: finite values have actual-walk witnesses and are no greater than the corresponding buffered candidates, so negative diagonals cannot be missed. Different update details can produce different temporary negative values.

Negative-cycle detection and affected pairs (continued)

Compute reachability R separately. Mark(i, j) as−∞ exactly when some k satisfies D[k, k]<0, R[i, k], R[k, j]. Sufficiency follows by entering a negative cycle, repeating it arbitrarily, and leaving toward j. For necessity, if the i-to-j relevant region has no negative cycle, deleting all nonnegative cycles leaves one of finitely many simple paths with a finite lower bound. Its distance cannot be−∞. Unreachable pairs remain∞; other Floyd entries remain valid.

Mark affected pairs before returning results, and do not feed−∞ back into ordinary∞ arithmetic. Johnson may reject the whole graph on any negative cycle; partial finite results require these pairwise semantics.

A negative cycle need not affect the whole graph

Use1→2:2,2→3:−3,3→2:1,3→4:4, with isolated5. Cycle 2→3→2 costs−2.

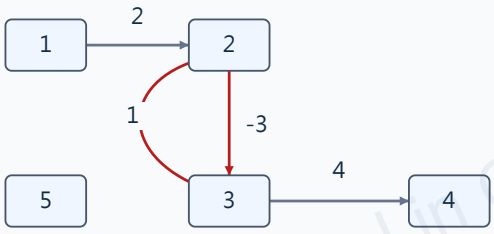
Source / destination	1	2	3	4	5
1	0	−∞	−∞	−∞	∞
2	∞	−∞	−∞	−∞	∞
3	∞	−∞	−∞	−∞	∞
4	∞	∞	∞	0	∞
5	∞	∞	∞	∞	0

Vertex 1 can enter the cycle and leave for 4, giving $\delta(1,4)=-\infty$. There is no return to 1, so $\delta(1,1)=0$. Vertices 4,5 cannot enter the cycle and retain zero self-distances. A negative diagonal does not justify marking every matrix cell as erroneous.

A negative cycle need not affect the whole graph (continued)

A negative cycle affects 9 pairs, not the whole matrix

Red cycle 2→3→2 costs−2. Red cells require both entry to and exit from the cycle.



	1	2	3	4	5
1	0	−∞	−∞	−∞	∞
2	∞	−∞	−∞	−∞	∞
3	∞	−∞	−∞	−∞	∞
4	∞	∞	∞	0	∞
5	∞	∞	∞	∞	0

Vertex 1 can enter but not return, so $\delta(1,1)=0$. Vertices 4,5 cannot enter.

Transitive closure uses the same split with Boolean algebra

Initialize $R^{(0)}[i, j]=(i=j)$ or an edge $i \to j$, including zero-length paths and true diagonals. Replace min/+ by OR/AND:

$R^{(k)}[i, j]=R^{(k-1)}[i, j]$ OR$(R^{(k-1)}[i, k]$ AND $R^{(k-1)}[k, j])$.

Remove repeated vertices from a path regardless of cycle weights and split by whether intermediate k appears. Both directions cover exactly reachability. This proof does not require absence of negative cycles. In-place updates are safe: R[k, k]=true and R[i, k] OR(R[i, k] AND true)=R[i, k], with the same argument for row k.

For the common example, final rows are[1,1,1,1],[0,1,1,1],[0,1,1,1],[0,1,1,1]. Time is $\Theta(n^3)$ Boolean operations, space $\Theta(n^2)$ Boolean values. Bit packing can save storage and exploit word parallelism, but requires an explicit word-size and implementation assumption; arbitrary-length bit strings are not single constant-time words.

25.3 Johnson’ s algorithm for sparse graphs

Johnson makes all edges nonnegative while preserving relative costs of routes with fixed endpoints. For any finite potential $h:V \to R$, define $\hat{w}(u, v)=w(u, v)+h(u)-h(v)$. Summing along $v_0, \ldots, v_r$ cancels internal potentials:

$\hat{w}(path)=w(path)+h(v_0)-h(v_r)$.

Every route with the same endpoints receives the same adjustment, preserving strict comparisons and ties, and therefore the **set** of shortest paths. Different tie-breaking can still select different predecessor trees. Cycles receive adjustment0, preserving all negative cycles. Reweighting cannot repair a negative-cycle problem.

Adding the same constant C per edge instead adds rC to an r-edge route and can change the optimum. For direct$1 \to 4$:6 and two-edge $1 \to 2$:2,$2 \to 4$:2, the original two-edge cost 4 beats6. Adding 3 per edge gives direct9 versus two-edge 10, reversing the result.

Constructing a potential and rejecting negative cycles

Add a super-source s with zero edges to all original vertices and no incoming edges. If the original graph has no negative cycle, neither does the augmented graph. Every vertex is reachable, so Bellman–Ford gives finite $h(v)=\delta(s, v)$. For each edge $h(v) \leq h(u)+w(u, v)$, equivalent to $\hat{w}(u, v) \geq 0$. This is the needed Dijkstra contract.

The n source edges need not be stored. Initialize all $h[v]=0$, as after relaxing them, and synchronously scan original edges for $n-1$ rounds, then test whether $h[v]>h[u]+w(u, v)$ still holds. This gives zero-cost starts at any original vertex and an $n-1$-edge original budget. Without negative cycles a simple shortest path suffices. If any negative cycle exists, its potential inequalities cannot all hold, since summing gives $0 \leq$ a negative number. The final scan must reject it.

In the common example, start $h=(0,0,0,0)$. Round 1 lowers only $h[3]$ to -2; rounds 2,3 do nothing. Six reweighted edges in input order are 4,9,0,6,1,1, all nonnegative. This h is not distance from original vertex 1: $h[2]=0$ whereas $\delta(1,2)=4$.

Constructing a potential and rejecting negative cycles (continued)

Johnson potentials and six reweighted edges

h[1]=0		h[2]=0	h[3]=-2	h[4]=0
Original edge	w	h[u]	h[v]	$\hat{w}=w+h[u]-h[v]$
1→2	4	0	0	4+(0)−(0)=4
1→3	7	0	-2	7+(0)−(-2)=9
2→3	-2	0	-2	-2+(0)−(-2)=0
2→4	6	0	0	6+(0)−(0)=6
3→4	3	-2	0	3+(-2)−(0)=1
4→2	1	0	0	1+(0)−(0)=1

Green weights are nonnegative. The potential uses a new zero-edge source, not original vertex 1.

Run Dijkstra per source, then restore original distances

```
Johnson(G):
  h=Bellman-Ford potential with all-zero initialization
  if any edge can still relax: report a negative cycle and stop
  for each original edge: ŵ(u, v)=w(u, v)+h[u]-h[v]
  for source u in V:
    run Dijkstra on original vertices with nonnegative ŵ, giving distHat
    for v in V:
      D[u, v]=∞ if distHat[v]==∞ else distHat[v]-h[u]+h[v]
  return D
```

From 1, distHat begins(0,∞,∞,∞). Extract 1 to get(0,4,9,∞); extract 2 to lower3 from 9 to 4 and assign4 value 10; extract 3 lowers4 to 5. Extract 4 cannot improve2. Extracted items are 1:0,2:4,3:4,4:5, using smaller IDs on distance ties. Final(0,4,4,5) restores to(0,4,2,5).

Other distHat rows are[∞,0,0,1],[∞,2,0,1],[∞,1,1,0]. Subtract source potential and add destination potential to recover the final Floyd matrix. For 3→2, this gives 2−(−2)+0=4; for 4→3, it gives 1−0+(−2)=−1. Unreachable entries stay∞; do not apply finite arithmetic to a sentinel.

Run Dijkstra per source, then restore original distances (continued)

Dijkstra on nonnegative weights and distance restoration

All reweighted edges are nonnegative. Cells show distHat; smaller IDs break ties.

Action	To 1	To 2	To 3	To 4
Initialize	0	∞	∞	∞
Extract 1	0	4	9	∞
Extract 2	0	4	4	10
Extract 3	0	4	4	5
Extract 4	0	4	4	5

Green: strict improvement. Original distance = distHat − h[1] + h[v].
Final (0,4,4,5) restores to (0,4,2,5), matching Floyd row1.

Sparsity, queues, and the unavoidable output cost

All-zero potential initialization costs $O(n)$, plus $n-1$ scans and a final check of m original edges, giving $O(nm+n)$. Generic Bellman–Ford on the explicitly augmented graph costs $O((n+1)(m+n))$; include its n added edges.

Fibonacci Dijkstra takes $O(m+n \log(n+1))$ amortized. Running it n times and filling n^2 outputs gives $O(nm+n^2 \log(n+1))$. Binary heaps give $O(n(m+n)\log(n+1))$; their log factor on m does not disappear. Graph and per-source work use $O(n+m)$, plus $\Theta(n^2)$ if the full output is retained. Row streaming can reduce resident output space but does not reduce the n^2-entry output itself.

For $n=1000$, $m=5000$, Floyd performs10^9 triple-index updates. Potential computation scans at most about 5×10^6 original edges. Binary repeated-Dijkstra's leading expression is $1000 \cdot 6000 \cdot \log_2 1000 \approx 5.98\times10^7$; Fibonacci's is $5\times10^6+10^6 \cdot \log_2 1000 \approx 1.50\times10^7$. These count different modeled operations and are **not measured execution times or directly divisible speedups**. For dense $m=\Theta(n^2)$, Floyd remains simple and $\Theta(n^3)$, and Johnson's sparse-graph advantage disappears.

References

Bibliography

Chapter source

Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 3rd ed., MIT Press, 2009, Chapter 25.

Lin Cing
https://github.com/xinhi6716/computer-science-self-study

References and attribution

Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, Clifford Stein. Introduction to Algorithms, 3rd Edition.

Original educational synthesis by Lin Cing. Original material is licensed under CC BY 4.0. The referenced textbook is not distributed with these notes.

https://github.com/xinhi6716/computer-science-self-study

Lin Cing
https://github.com/xinhi6716/computer-science-self-study